FULL FLOW WITH EXAMPLE

🧑 Customer: Arjun from Bangalore

Product: Leather Wallet (₹2,499)

Chooses: COD

1️⃣ Order Placed

Arjun clicks “Place Order (COD)”

👉 Backend detects:

New user

COD

Medium order value

System triggers verification

2️⃣ COD Verification (via MSG91)

Step 1: OTP SMS

“Your OTP is 4821. Enter to confirm your order.”

Arjun enters OTP → verified ✅

Step 2: WhatsApp Confirmation

“Hi Arjun, please reply YES to confirm your order for Leather Wallet (₹2,499)”

Arjun replies: YES

👉 Order now moves forward

👉 If no reply → auto cancel in 2 hours

3️⃣ Order Confirmation

System confirms order

WhatsApp message:

“Your order is confirmed ✅

Leather Wallet

Expected delivery: 3–5 days

We’ve started preparing it.”

👉 Customer feels secure → reduces cancellation

4️⃣ Shipment Creation (via Delhivery)

Backend auto:

Creates shipment

Generates AWB: DL123456789

Schedules pickup

👉 No human involved

5️⃣ Shipping Updates (Auto WhatsApp)

When shipped:

“Your order has been shipped 🚚

Track here: [link]”

Out for delivery:

“Out for delivery today. Keep ₹2,499 ready.”

👉 This reduces failed deliveries massively

6️⃣ Delivery Done

Delhivery updates status → webhook triggers

WhatsApp:

“Delivered ✅

How’s your new wallet? Reply with feedback!”

7️⃣ Post-Delivery Profit Flow

Day 2:

“Hope you’re loving it. Mind sharing a quick review?”

Day 5:

“Leather tip: Keep it dry & conditioned for long life.”

👉 This builds brand, not just sales

Day 10 (Upsell):

“Complete your set 👇

Matching leather belt — 10% off for you”

8️⃣ What If Something Breaks?

Let’s say:

❌ Delhivery API fails during shipment creation

👉 System does:

Retry automatically

Log error

Alert admin

No lost orders. No chaos.

🔁 Now Compare: Prepaid Example

Same customer, but pays via Razorpay

Flow difference:

No OTP needed

Payment webhook confirms instantly

Order moves faster

👉 Bonus: You can prioritize prepaid orders in shipping

(VINTAGE HIDES)

1️⃣ Order Placement

Customer lands → selects product → enters details

Decision split:

Prepaid → via Razorpay

COD → goes into verification flow

2️⃣ Payment + COD Verification

Prepaid Flow:

Payment processed via Razorpay

Webhook confirms payment

Order marked PAID

COD Flow:

OTP sent via MSG91

Customer verifies OTP

WhatsApp message:

“Reply YES to confirm your order”

👉 No reply / no OTP = auto cancel

👉 High-risk orders:

Force prepaid OR reject

3️⃣ Order Confirmation Trigger

Once order is valid:

WhatsApp confirmation sent

Order created in system

Message includes:

Product details

Delivery estimate

Brand reassurance

4️⃣ Shipment Creation (Auto)

Using Delhivery

AWB generated automatically

Pickup scheduled

Tracking ID stored

👉 Zero manual work

5️⃣ Real-Time Tracking Engine

Delhivery → webhook → your backend reacts

Status updates trigger WhatsApp:

Shipped

In transit

Out for delivery

Delivered

👉 If delay detected:

Auto message: “Slight delay, we’re on it”

Customer should never ask for updates.

6️⃣ Failure Handling Layer (Silent Money Saver)

System auto-detects:

Payment success but no order → retry + alert

Shipment creation failed → retry

WhatsApp failed → fallback SMS

👉 Every failure is:

Logged

Retried

Alerted

No silent errors. Ever.

7️⃣ COD Risk Engine (Profit Protection)

Before confirming COD:

Check past behavior

Check pincode risk

Check order value

Actions:

Block COD

Force prepaid

Allow with OTP

👉 Repeat RTO customer = COD disabled

8️⃣ Delivery → Post-Purchase Engine

Once delivered:

Immediate:

WhatsApp: “Delivered — how is it?”

After 1–2 days:

Review request

After 5–7 days:

Care tips (leather maintenance = brand building)

After 10–15 days:

Upsell (wallets, belts, etc.)

9️⃣ Abandoned Cart Recovery

Triggered if no checkout:

30 mins → reminder

6 hours → urgency

24 hours → offer

👉 This alone can recover 10–20% revenue.

🔟 Win-Back System

No purchase in 30–45 days:

WhatsApp:

New drop

Limited stock

Personalized tone

⚙️ SYSTEM ARCHITECTURE (Simple View)

User Action → Backend → Webhook → Trigger → Automation

Razorpay → Payment Confirm → Order Created

MSG91 → Messaging Layer

Delhivery → Shipping + Tracking

Everything connected. Everything automated.

“Show me webhook handling for all 3 systems”

“Where do we track failures?”

“How are COD orders filtered?”

“Show me automation flows, not just integrations”

1️⃣ Webhook Handling (Core Backbone)

Using:

Razorpay

Delhivery

MSG91

What “correct” looks like:

Razorpay Webhook

Event: payment.captured

Backend verifies signature

Order marked as PAID

Triggers:

Order confirmation (WhatsApp)

Shipment creation

Delhivery Webhook

Event: shipment status update

Backend updates order status:

Shipped / Out for delivery / Delivered / RTO

MSG91 Webhook (optional but strong)

Message delivered / failed tracking

👉 What you should ask:

“Show me webhook logs”

“What happens if webhook fires twice?”

“How do we prevent duplicate orders?”

If they don’t handle idempotency → messy system.

2️⃣ Failure Tracking System (This separates pros from amateurs)

You need a central error log system

Must track:

Payment success but order not created

Shipment creation failed

WhatsApp message failed

API timeouts

What your dev should show:

A dashboard or logs with:

Timestamp

Order ID

Error type

Retry status

👉 Example:

Order #1245

Error: Delhivery API failed

Retry: Pending

Status: Alert sent to admin

👉 What matters:

Auto retry (at least 2–3 attempts)

Admin alert (email or WhatsApp)

If errors disappear silently → you lose money without knowing.

3️⃣ COD Filtering Logic (Your profit shield)

This is non-negotiable in India.

Your system should do this BEFORE confirming order:

Step 1: Check customer history

Past RTO? → COD blocked

Step 2: Order value

High value (> ₹3–5k)? → force prepaid or partial payment

Step 3: Location risk

Certain pincodes → COD disabled or verified

Step 4: OTP verification via MSG91

No OTP = no order

Example Logic:

IF (new user AND COD AND order > ₹3000)

→ Require prepaid OR OTP + manual confirmation

IF (past RTO customer)

→ Disable COD

IF (high-risk pincode)

→ COD not available

👉 Ask your dev:

“Where is this logic implemented?”

“Can we update rules without code changes?”

If answer is “hardcoded” → not scalable.

4️⃣ Automation Flows (This is where revenue is made)

Not “messages.” Full flows.

A. Order Confirmation Flow

Trigger: Order success

Message:

Order details

Delivery timeline

Trust signal

B. COD Verification Flow

Trigger: COD order placed

Flow:

OTP via MSG91

WhatsApp message:

“Reply YES to confirm your order”

👉 No response → auto cancel after X hours

C. Shipping Flow

Trigger: Delhivery updates

Messages:

Shipped

Out for delivery

Delivered

D. Abandoned Cart Flow

Trigger: No checkout after X minutes

Flow:

Reminder (30 min)

Reminder + urgency (6 hours)

Offer (24 hours)

E. Post-Delivery Flow (Most important for profit)

Trigger: Delivered

Flow:

“Your order delivered — how is it?”

Review request

Care tips (build brand)

Upsell after 5–7 days

F. Win-back Flow

Trigger: No purchase in 30–45 days

Message:

New drop / limited stock

Personal tone, not spam

1. Predictive COD/RTO scoring

Not just:

“High risk pincode → block”

But:

Probability scoring per order

Dynamic decisions

2. Customer segmentation engine

Different behavior for:

First-time buyers

Repeat customers

High-value buyers

👉 Right now, everyone gets similar flows = missed upside

3. Funnel optimization layer

You should be tracking:

Where users drop off

COD vs prepaid conversion

WhatsApp click rates

👉 Then adjusting flows based on data

4. Operational dashboard (real-time)

You should be able to see instantly:

Orders stuck

Failed payments

Shipment delays

RTO trends

👉 Without this, you’re still reacting late

COMPLETE SYSTEM)

We’ll run this as a live scenario + logic behind it

🧑 Example Customer

Name: Arjun

Location: Bangalore

Product: Leather Backpack (₹5,999)

Device: Mobile

Source: Instagram ad

1️⃣ Smart Checkout Layer

Before order is even placed:

👉 System captures:

Device type

Location

Order value

User history

🎯 Decision Engine (Real Upgrade)

Instead of fixed rules:

Risk Score = f(user history + pincode + order value + behavior)

👉 Outcome:

Low risk → allow COD

Medium risk → OTP required

High risk → prepaid only

2️⃣ Payment / COD Flow

Prepaid (via Razorpay)

Webhook confirms payment

Order instantly approved

COD (via MSG91)

Step 1: OTP

“Your OTP is 7392”

Step 2: WhatsApp

“Reply YES to confirm your ₹5,999 order”

👉 Smart twist: If user delays:

“Only limited stock left — confirm now”

3️⃣ Dynamic Order Confirmation

Not same message for everyone.

First-time buyer:

“Welcome to VINTAGE HIDES — your order is confirmed”

Repeat buyer:

“Good to have you back — your next piece is on the way”

👉 Personalization = higher trust

4️⃣ Intelligent Shipping System (via Delhivery)

Auto shipment creation

AWB generated

Courier assigned

Smart routing:

High-value orders → faster shipping

Risky COD → extra confirmation

5️⃣ Predictive Tracking + Communication

Instead of reacting:

👉 System predicts delays

If delay likely:

“Slight delay expected — we’re monitoring it”

If delivery today:

“Out for delivery — please be available”

👉 Result:

Fewer failed deliveries

Lower RTO

6️⃣ Real-Time Failure Intelligence

Every failure is handled instantly:

Payment mismatch → auto reconcile

Shipment fail → retry + switch courier

Message fail → fallback SMS

👉 Plus: Admin gets alert:

“Order #1452 needs attention”

7️⃣ Delivery + Experience Layer

On delivery:

“Delivered ✅ — how does it feel?”

Based on response:

Happy customer →

Ask review

Offer referral

No response →

Send reminder

Offer small incentive

8️⃣ Post-Purchase Revenue Engine

Day 3:

Leather care tip (build authority)

Day 7:

Cross-sell: “Your backpack pairs well with this wallet”

Day 21:

Repeat trigger: “Your leather ages better with use — explore more”

👉 This is where profits scale.

9️⃣ Abandoned Cart (Smart Version)

Not generic reminders.

Based on behavior:

Viewed 1 product → simple reminder

Viewed multiple → comparison angle

High-value cart → urgency + trust

🔟 Self-Learning System (THIS = 10/10)

This is the difference.

System continuously tracks:

COD success rate

RTO % by pincode

Conversion by traffic source

WhatsApp engagement

Then auto-adjusts:

High RTO area → COD disabled

High prepaid area → push prepaid offers

High-value customers → VIP treatment